Unit 8: Web Application Deployment 4 hours

Compiled By

Lecturer, Sunil Bahadur Bist

NAST, Dhangadhi

Contents

- Introduction to Deployment
- Deployment platforms
- Basic differences between development and production environments.
- Setting up environment variables and basic production settings.
- Introduction to using Gunicorn to serve Flask apps.
- Setting up basic HTTPS using a free certificate (e.g., Let's Encrypt).
- Brief introduction to monitoring and logging.
- Importance of security in deployment.

Introduction to Deployment

- Deployment is the process of publishing or releasing a software application from development into a live server or environment where users can access and use it.
- It involves setting up the necessary infrastructure, configuring the system, and ensuring the application runs smoothly in production.
- Deployment is a critical step that makes software available for real-world use.

Deployment Platform

- A **deployment platform** is a system or environment used to deliver, install, configure, and manage software applications for end users.
- It serves as the bridge between software development and actual use, ensuring that applications run reliably and efficiently on target systems.
- Deployment platforms handle various tasks such as:

Deployment Platform

- Hosting the application (locally, on-premises, or in the cloud)
- Managing updates and version control
- Providing scalability and fault tolerance
- Monitoring and logging performance and usage
- Securing access and protecting data

Deployment Platform

- Depending on the project requirements, different types of deployment platforms can be used, including:
  - **Cloud-based platforms** (e.g., AWS, Microsoft Azure, Google Cloud Platform)
  - **Container-based platforms** (e.g., Docker, Kubernetes)
  - **Platform-as-a-Service (PaaS)** providers (e.g., Heroku, Firebase)
  - **On-premise servers** (hosted within a local network)

Flask Deployment Platform

- A **deployment platform for Flask** refers to the environment or service where your Flask application is hosted and made available to users.
- It includes the infrastructure, web server, and tools required to run and manage the app in production.
- When developing web applications using **Flask**, a lightweight Python web framework, deploying the app to a suitable **deployment platform** is essential to make it accessible to users over the internet or within a network.

Why to Deploy a Flask App?

- To make your application accessible to users
- To run the app in a stable and secure production environment
- To scale the app as user demand increases
- To integrate services like databases, caching, monitoring, etc.

Flask Deployment Platforms

- **Local Server / Development Server**
  - Suitable only for development and testing.
  - Not secure or efficient for production.
- **WSGI Servers (Production-ready)**
  - Examples: **Gunicorn**, **WSGI**
  - Flask needs to be served by a WSGI-compliant server for production use.

Flask Deployment Platforms

- **Cloud Platforms**
  - **Heroku**: Easy to use for beginners; supports Git-based deployments.
  - **PythonAnywhere**: Simple for hosting small Flask apps.
  - **Google Cloud Platform (App Engine)**, **AWS Elastic Beanstalk**, **Microsoft Azure**: Scalable cloud solutions.
- **Containers**
  - **Docker**: Package Flask apps into containers for consistent deployment across environments.
  - Often combined with **Kubernetes** for orchestration.

Flask Deployment Platforms

- **Virtual Private Server (VPS)**
  - Services like DigitalOcean or Linode allow full control of the deployment environment.
- **Platform-as-a-Service (PaaS)**
  - Handles infrastructure, scaling, and deployment for you. Examples: Heroku, Render, Fly.io

Flask Deployment Stack

- **Flask app** (application code)
- **Gunicorn** or **WSGI** (WSGI server)
- **Nginx** (reverse proxy and static file server)
- **Systemd** or **Supervisor** (process management)
- **PostgreSQL / MySQL / SQLite** (database)

Flask Deployment

- In summary, a deployment platform in Flask is the foundation for making your app usable in real-world scenarios.
- Selecting the right one depends on your project's needs, complexity, budget, and scalability requirements.

Difference Between Both Environment

- In web application development, including Flask apps, understanding the difference between **development** and **production** environments is crucial for building reliable, secure, and scalable applications.

Feature	Development	Production
Purpose	Used for writing, testing, and debugging code	Used to run the app live for end users
Debug Mode	Enabled (shows detailed error messages)	Disabled (hides sensitive error details)
Performance	Not optimized; may be slower	Optimized for speed and efficiency

Difference Between Both Environment

Feature	Development	Production
Security	**Basic security, relaxed settings**	**Strict security settings, hardened configuration**
Logging	Verbose and detailed logs for debugging	Concise logs focused on errors and performance
Error Handling	Displays stack traces and debug info	Shows user-friendly error pages only
Database	May use a lightweight DB like SQLite	Uses production-grade DBs like PostgreSQL or MySQL
Deployment Tools	Simple (e.g., Flask's built-in server)	Production-ready tools (e.g., Gunicorn + Nginx)
Access	Typically local (localhost)	Publicly accessible via domain/IP
Environment Variables	May use defaults or .env files	Securely managed with environment variables or secret managers

Difference Between Both Environment

Example,

- In a **development environment**, we run your app as:

```
flask run
```

- In a **production environment**, we have to use a production WSGI server:

```
gunicorn app:app
```

- And make sure to disable debug mode false in your code:

```
app.run(debug=False)
```

Setting up environment variables and basic production settings.

Environment Variables

Environment variables store configuration values outside the code, such as:

- **FLASK_ENV=production**
- **SECRET_KEY=your-secret-key**
- **DATABASE_URL=mysql://...**

These make your app:

- **More secure**
- **Easier to configure**
- **Environment-agnostic (dev, staging, production)**

Setting up environment variables and basic production settings.

Creating a .env File (Local or Docker)

Create a file named .env in your project root:

```
FLASK_ENV=production
FLASK_APP=app.py
SECRET_KEY=super-secret-key
DATABASE_URL=mysql+pymysql://user:pass@localhost/dbname
```

Don't commit this to Git. Add .env to .gitignore.

Loading Environment Variables in Flask

Install the python-dotenv package:

```
pip install python-dotenv
```

Setting up environment variables and basic production settings.

Basic Production Settings for Flask

```
# config.py
class Config:
    SECRET_KEY = os.getenv("SECRET_KEY")
    DEBUG = False
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    SESSION_COOKIE_SECURE = True  # for HTTPS
    REMEMBER_COOKIE_SECURE = True
    PREFERRED_URL_SCHEME = 'https'
```

Then load it:

```
app.config.from_object('config.Config')
```

Gunicorn to Serve Flask Application

- Gunicorn (Green Unicorn) is a Python WSGI HTTP server commonly used to deploy Flask applications in production.
- It acts as a bridge between your Flask app and a web server (like Nginx), efficiently handling multiple requests concurrently.

Why Use Gunicorn?

- **Production-grade:** Unlike Flask's built-in development server (flask run), Gunicorn is designed for production use.
- **Concurrency support:** Handles multiple simultaneous requests using worker processes or threads.
- **WSGI compliant:** Works with any Python WSGI-compatible framework (like Flask or Django).

Installing Gunicorn & Serving Flask App

- You can install Gunicorn using pip:

  pip install gunicorn

- Serving a Flask App with Gunicorn

  Assume you have a Flask app structure like this:

  appname/
  |— app.py
  |__ templates
  |__ static
  |__ requirements.txt

Installing Gunicorn & Serving Flask App

- And app.py contains

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello Learners from Flask via Gunicorn WSGI Server!"
```

Installing Gunicorn & Serving Flask App

- Run the app using Gunicorn:

  gunicorn app:app

- Here, app:app : The first app is the filename (app.py), the second app is the Flask application instance inside that file.

- To deploy flask application to pythonanywhere follow:

https://www.youtube.com/watch?v=XU4PmRXJUVQ&ab_channel=ProgrammingKnowledge

Setting up basic HTTPS using a free certificate (e.g., Let's Encrypt)

- To set up **basic HTTPS** using a free certificate from **Let’s Encrypt**, you can use **Certbot**, a tool that automates the process.
- Here is an step-by-step guide for setting up HTTPS for your web application (e.g., Flask app) deployed on **Ubuntu Linux** with **Nginx** (also works with Apache).

Setting up basic HTTPS using a free certificate (e.g., Let's Encrypt).

Prerequisites

- A domain name (e.g., example.com)
- A public-facing server (e.g., VPS like DigitalOcean, AWS EC2)
- Access to the server via SSH
- Nginx or Apache installed and running

Step-by-Step Setup with Certbot and Nginx

Install Certbot On Ubuntu (22.04 or newer)

sudo apt update

sudo apt install certbot python3-certbot-nginx

Configure Your Nginx Server Block

Example /etc/nginx/sites-available/example.com:

Step-by-Step Setup with Certbot and Nginx

```
server {
    listen 80;
    server_name example.com www.example.com;

    location / {
        proxy_pass http://localhost:5000;  # Flask app
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Step-by-Step Setup with Certbot and Nginx

Then enable the config:

sudo ln -s /etc/nginx/sites-available/example.com /etc/nginx/sites-enabled/

sudo nginx -t

sudo systemctl reload nginx

Obtain an SSL Certificate Using Certbot

Run:

sudo certbot --nginx -d example.com -d www.example.com

Step-by-Step Setup with Certbot and Nginx

- **Certbot will:**
  - Communicate with Let’s Encrypt
  - Automatically configure your Nginx to use HTTPS
  - Reload Nginx
- **You will see:** *"Congratulations! Your certificate and chain have been saved at..."*

Step-by-Step Setup with Certbot and Nginx

- **Test Auto-Renewal**
- Let’s Encrypt certificates expire every **90 days**, so enable auto-renewal:
- **Check with:**
  - sudo certbot renew --dry-run
- Certbot automatically sets a cron job or systemd timer for renewal.

Brief introduction to monitoring and logging

- Monitoring and logging are essential practices in software development and deployment, especially for maintaining system health, diagnosing issues, and ensuring performance and security.
- **Logging**
  - Logging involves recording events that happen during the execution of a program. These logs can include error messages, system warnings, user actions, API calls, database queries, etc.
  - Example tools: Python’s logging module, Log4j (Java), ELK Stack (Elasticsearch, Logstash, Kibana).

Brief introduction to monitoring and logging

- **Monitoring**
  - Monitoring involves continuously observing a system to track its health, performance, and behavior. It helps in detecting issues (like downtime or high latency) in real-time and triggering alerts.
  - Example tools: Prometheus, Grafana, Datadog, Nagios.
- Together, monitoring and logging provide visibility into how applications perform and help identify and resolve issues faster.

Importance of security in deployment

- Security during deployment is crucial because a small misconfiguration or vulnerability can lead to major data breaches, system downtime, or unauthorized access.
- Here are the key reasons why it is important:
  - **Protecting sensitive data:** Secure deployment ensures that user data, credentials, and APIs are not exposed to attackers.
  - **Preventing unauthorized access:** Implementing authentication, authorization, and secure protocols (e.g., HTTPS) during deployment safeguards systems from intrusion.

Importance of security in deployment

- **Mitigating vulnerabilities:** Proper patching, use of secure dependencies, and environment hardening reduce the attack surface.
- **Compliance and regulations:** Many systems must meet legal or organizational security standards (e.g., GDPR, HIPAA).
  - GDPR (General Data Protection Regulation)
  - HIPAA (Health Insurance Portability and Accountability Act)
- **Building trust:** A secure deployment assures users that the application is reliable and their data is safe

Security Factors During Deployment

- When deploying a Flask application to a production environment, **security** becomes a top priority.
- Failing to secure your app can expose it to various threats such as data breaches, SQLi, code injection, or denial-of-service attacks.
- Here are the **key security factors** to consider during deployment:

Security Factors During Deployment

- **Disable Debug Mode**
  - **Why:** Flask's debug mode shows detailed error messages, including stack traces and environment variables a major security risk.
  - **How:**

    ```
    app.run(debug=False)
    ```

- **Use a Production WSGI Server**
  - Do not use flask run in production.
  - Use a production-grade WSGI server like Gunicorn or uWSGI behind a reverse proxy like Nginx.

Security Factors During Deployment

- **Environment Variables & Secrets Management**
  - Store sensitive data (e.g., API keys, database passwords) securely using:
    - .env files (with caution)
    - Secret managers (e.g., AWS Secrets Manager, environment variables)
  - Avoid hardcoding secrets in your code.
- **HTTPS / SSL Encryption**
  - Use HTTPS to encrypt communication between the client and server.
  - Set up SSL certificates using tools like Let's Encrypt.
  - Redirect HTTP to HTTPS in your web server configuration (e.g., Nginx).

Security Factors During Deployment

- **Input Validation and Sanitization**
  - ***Always validate and sanitize user inputs to prevent:***
    - SQL Injection
    - Cross-site Scripting (XSS)
    - Cross-site Request Forgery (CSRF)
  - ***Use Flask extensions like:***
    - Flask-WTF for form validation and CSRF protection
    - ORM tools like SQLAlchemy to prevent SQL injection

Security Factors During Deployment

- **Secure Headers**
  - Add HTTP security headers like:
    - Content-Security-Policy
    - X-Frame-Options
    - X-XSS-Protection
  - Use Flask extensions like Flask-Talisman to easily apply these headers:

    ```
    from flask_talisman import Talisman
    Talisman(app)
    ```

Security Factors During Deployment

- **Access Control and Authentication**
  - Implement proper authentication (login system) and authorization (user roles/permissions).
  - Use libraries like:
    - Flask-Login for user session management
    - Flask-JWT or OAuth for token-based authentication

Security Factors During Deployment

- **Limit File Uploads and Types**
  - Restrict file upload types and sizes to prevent malware or server overload.
  - Validate file extensions and MIME types.
  - Store uploaded files outside the root directory if possible.
- **Logging and Monitoring**
  - Log important events (e.g., failed login attempts, errors).
  - Monitor server activity to detect unusual or malicious behavior.
  - Use tools like Sentry, ELK stack, or cloud monitoring services.

Security Factors During Deployment

- **Keep Dependencies Updated**
  - Regularly update Flask and its extensions to patch known vulnerabilities.
  - Use tools like:

    pip list --outdated

    pip-audit (for checking vulnerabilities) install it before use.

    pip install –upgrade <packagename> : to remove known vulnerabilities

Security Factors During Deployment

Security Factor	Recommendation
Debug Mode	Disable in production
WSGI Server	Use Gunicorn or uWSGI
HTTPS	Use SSL certificates
Input Sanitization	Validate user input
Secrets Handling	Use environment variables securely
Secure Headers	Add via Flask-Talisman
Authentication & Authorization	Implement proper access control
File Uploads	Restrict types and size
Logging & Monitoring	Log events and monitor performance
Package Management	Keep dependencies updated

End

- Best wishes for your exam
- For any queries email at sunil@nast.edu.np